UAV OPERATION

CUAV V5+ flight controller calibration

JANUARY 7, 2025

PRAVEEN CHAMIKARA

Contents

Goals 2
Flight controller 2
Powering up the flight controller 2
Additional features of the V5+ 3
Calibrating the flight controller 3
Selecting and setting up the airframe 3
Receiver input from transmitter calibration with the flight controller 3
Flight mode setup 4
Power setup 4
Sensor setup 4
Procedure 6
Errors encountered and how I tackled them 7
TO BE CONTINUED 9

Goals

The main goals here are,

- Testing if the flight controller is compatible with the QGroundControl software
- Ensuring the new PX4 version supports the VTOL airframe and inbuilt firmware
- Ensuring all the Sensor calibrations happen smoothly
- Ensure that the flight controller communicates properly with the GCS
- Calibrating transmitter inputs into the flight controller system via SBUS protocols
- Connecting the telemetry module to the flight controller and ensuring wireless calibrations and communication could occur between the GCS and the flight controller
- Connect and setup the GPS module to the flight controller
- Add additional sensors to the flight controller via I2C protocols

Flight controller

This here is the Pixhawk V5+ flight controller by CUAV. This is widely used for UAVs (Unmanned Aerial Vehicles). This has many features, such as I2C to operate other devices, GPS, UART, ADC and many more! Im excited to work on this.

Powering up the flight controller

This flight controller has two power input ports (Power 1 and Power 2 are considered the main power inputs for this). We have to ensure that the power received matches the controller's requirement, which is around 5 V DC. To ensure we have backup power, we connect another power source to port “Power 2”.

Additional features of the V5+

There are a lot of I/O parts to ensure advanced UAV functionality. We have PWM ports (M1 – M8). We can connect and control servo motors to control the control surfaces. I believe the receiver input is received by this flight controller and further controlled by the flight controllers to stabilize the aircraft. We have PPM/SBUS ports to receive receiver inputs. GPS ports to connect GNSS modules for navigation. I2C and CAN ports are further used to connect and receive values from slave devices such as range finders, airspeed sensors and other compatible devices.

Telemetry ports are used for ground communication purposes. The USB port is used for computer input, such as firmware updates and debugging.

Calibrating the flight controller

We are supposed to use ground control station software (GCS). QGroundControl is a software that can used to calibrate the flight controller. Now, this over here is a way more time-saving option than manually coding the firmware side for the flight controller. This software has the best firmware for the airframes. Under airframes, we have many options, but for the sake of the project, we will go with a VTOL design. I have to know exactly what type of requirement is accepted from our design, and based on that, I will select a suitable airframe design. So here's a detailed calibration procedure I found in an online tutorial.

First of all, connect the flight controller to the computer. Then, go to flight settings in QGroundControl, and we will see a summary. This shows all the necessary parts that we should look into. Thanks to the flight controller feature to add a camera and the QGroundControl software, it's much easier for us to integrate such systems.

Next, we will have to disconnect the flight controller and wait till it asks to reconnect it. Once we reconnect the flight controller, the firmware will be updated for the controller. Select the PX4 Pro.

Selecting and setting up the airframe

Now, after the download, go to the summary, and you will see red dots in some categories, meaning you will have to look into them. Now that we have it downloaded, we can move it into the airframe. Get some ideas from the scientist tomorrow to understand the requirements and choose a proper airframe based on that. Then, apply and restart the setup, and you will have to restart your drone. After the process is done, leave it for a little time till the process is done. Once it is complete, you will see a green light under the airframe section, indicating that the airframe is clear.

Receiver input from transmitter calibration with the flight controller

After the airframe is complete move into the radio section and under this and make sure to get your transmitter connected to this. After ensuring that the throttle sticks are centred, start the calibration process by clicking calibrate. Do all the things with the throttle sticks as they show. Click next after each consecutive movement. Repeat the same for your switches as well. Now the radio section is complete!

Flight mode setup

We can use this to set our switches to assigned channels. Set the mode channel, arm switch and emergency kill switch for safety reasons.

Power setup

Here, you are supposed to specify the source of power. This basically means the power supply for the drone. We will set it to Power module, and under the section of adding several cells in series, check the battery being used and find out how many cells are present in it. Later, check for the rest of the categories, such as the empty voltage, Full voltage and voltage divider, through the battery's datasheet and hit calculate. (Do further research to see why this is important.)

Sensor setup

This is one of the most important parts of the calibration process and can be tricky. The first thing we should check is the **compass.** Any external sensors can also be set up if our setup has any external sensors. So, I will look into the compass…

A compass in a flight controller is an **electronic sensor** used to measure the Earth's magnetic field and determine the vehicle's heading (direction relative to magnetic north). It plays a crucial role in navigation and stability, especially for autonomous or GPS-assisted flight. Here’s an overview of its purpose and functions. It's better to understand the process in detail instead of simply making it by looking at tutes. So, understand how the compass calibration works and affects the drone's functionality.

Heading Determination

- The compass provides the flight controller with the vehicle's orientation (yaw angle) relative to the magnetic north.
- This allows the flight controller to understand which direction the vehicle is facing.

Navigation

- Combined with GPS data, the compass helps the vehicle follow waypoints and navigate accurately to a destination.
- It ensures the vehicle maintains a proper heading even during long-distance flights.

Position Hold and Return-to-Home (RTH)

- For features like position hold and return-to-home, the compass works with GPS to ensure the vehicle flies in the correct direction.

Flight Stability

- The compass aids in maintaining stable flight by providing orientation data, particularly in windy conditions or when external forces act on the vehicle.

Simulation of Inertial Measurements

- When combined with the gyroscope and accelerometer (IMU), the compass helps correct drift in orientation readings over time.

Select a compass, place your drone or flight controller on a flat surface, and have **Rotation None.** Do everything it asks you to do. After completing the process, click reboot the vehicle, go to the firmware section, and do as you did earlier.

The next sensor we will deal with is the gyroscope. This is the most important part of a drone as it focuses on the alignment of the drone when faced with imbalanced motivating forces. Every drone should have one, and our VTOL should do so, too! Place the flight controller on a flat surface and click OK. Wait till the process is complete. The flight controller makes the current place its normal operating alignment. Whenever an imbalance force hits the drone, causing it to change alignment, the flight controller will create a counter output to realign the drone.

Next, look into the accelerometer. Understanding what goes on here is recommended, so I researched this online.

An **accelerometer** in a flight controller measures the forces of acceleration acting on the vehicle in three axes: **X, Y, and Z**. It provides critical data about the vehicle's orientation, movement, and stability. Here's how it contributes to the functionality of a flight controller:

1. **Orientation and Leveling (Pitch, Roll, and Tilt)**
    - The accelerometer measures the force of gravity, which helps the flight controller determine the vehicle's orientation relative to the Earth's surface.
    - This data ensures the vehicle stays level or maintains a specified tilt angle.
2. **Motion Detection**
    - The accelerometer detects linear acceleration along the three axes. This helps the flight controller identify if the vehicle is speeding, slowing, or maintaining steady motion.
3. **Stabilization**
    - During flight, the accelerometer works with the gyroscope to correct deviations in pitch and roll, ensuring smooth and stable flight.
    - It helps the vehicle recover quickly from disturbances like wind or sudden movements.
4. **Altitude Estimation (Vertical Acceleration)**
    - The Z-axis data (vertical acceleration) can help in estimating altitude changes, particularly in drones without barometers or for additional redundancy in altitude hold.
5. **Crash Detection**
    - Sudden and extreme changes in acceleration (like a hard impact) can be detected by the accelerometer, allowing the flight controller to log the event or trigger safety protocols.
6. **Inertial Navigation System (INS)**
    - When combined with a gyroscope and sometimes a magnetometer, the accelerometer contributes to the Inertial Navigation System. This allows the flight controller to calculate the vehicle's position, velocity, and orientation over time.

The process is quite simple. You need to place the flight controller as they ask in the diagrams and get them completed.

Then go to level horizon and press OK, similarly, due to the orientations.

This rounds up the whole setup of our drone. First, rely on a quadcopter design and later do more research. Many potential errors will be experienced; hence, it's better to debug and update the document with the modifications.

Procedure

Starting with the receiver, I must understand how the receiver pins are wired to the flight controller. I found this video online explaining how flight controllers are wired to the receiver. Every flight controller has a dedicated receiver pin, which will be shown as SBUS or DSM2, RSSI or receiver. Unfortunately, my FS IA6B receiver is not with me. This receiver is the best option I can rely on, as it supports all PWM, SBUS and PPM! It sucks that I don't have the receiver with me right now. I have the FS–R6B with me tho. It supports only PWM signals, so I must figure out how to connect the PWM signal outputs to the flight controller. Luckily, the V5+ supports PWM inputs through its analog pins! Now, I will have to figure out how the connection is made between the flight controller and the receiver! Just like a normal plane, even a quadcopter will have a Throttle (left throttle stick up and down), Roll (right throttle stick going left and right), Pitch (up and down movement of the right throttle) and Yaw (left throttle stick left-right). I will have to check the transmitter's correct output channel. For this purpose, I will wire the flight controller and calibrate them accordingly. I have a good idea about the flight controller now! Basically, PPM pins are a life safer, but unfortunately, my receiver misses it! I could've controlled all 4 inputs with one single wire! The pins M1 – M8 are basically used for the outputs such as ESCs or servo motors. The pins A1 to A6 are used for the analog inputs if we use PWM. I will have to check the respective channels for the Throttle, Roll, Pitch and Yaw and then connect them in order from A1 – A4. Later, I will have to set up the inputs from the QgGroundControl software and look into the servo outputs.

Here are the following channels for Throttle, Roll, Pitch and Yaw, respectively,

Channel 3 – Throttle

Channel 1 – Roll

Channel 2 – Pitch

Channel 4 - Yaw

I am planning to test a quadcopter design to evaluate the performance of the QGroundControl software's inbuilt firmware. I intend to conduct more research on VTOL systems, focusing on features like mode switching, airspeed sensor connections, and calibrations.

While I aim to complete the wiring and research for the VTOL as soon as possible, it's important to note that building a VTOL will not be as straightforward as constructing a standard quadcopter. The added complexity requires careful planning and execution.

Additionally, I need to explore GPS and telemetry ports to enable wireless communication and calibration. These features will allow the drone to execute real missions based on predefined flight paths, significantly enhancing its functionality.

I am eager to dive deeper into these areas and expand my understanding. So far, everything is progressing well!

After setting up the transmitter, I proceeded with the rest of the setup and ensured everything was ready for a test flight. However, I ran into an issue with the ESCs, which I've covered in the "Errors" section. To connect the ESCs properly, start by wiring the signal wire of the ESC to D14 and the ground to the ground to ensure the signal passes through. Once connected, power on your ESC. Next, open the BLHeli software to begin the interface setup. Select your board type, set the baud rate to 57600 (recommended), and choose the Arduino BLHeli bootloader. You can opt for the 4-way interface to calibrate all four ESCs at once, but this requires different wiring. Ensure the correct COM port is selected so that the Arduino can communicate properly with the software (I already have the necessary drivers for Arduino).

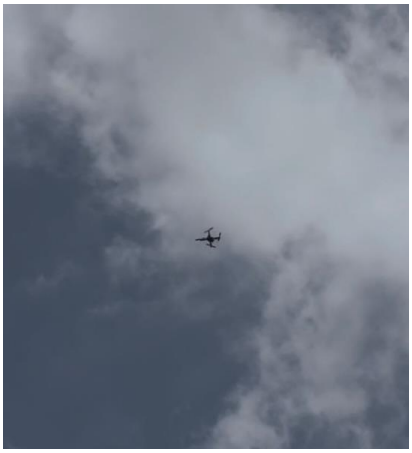
Click on the Arduino BLHeli bootloader, and the software will prompt you to select the BLHeli bootloader interface (USB/COM)—click "OK". Afterward, select the BLHeli interface and choose the USB/COM interface. Go to the ESC setup, click "Read Setup" to gather current ESC information, and then adjust the parameters to match the other three ESCs before clicking "Write Setup". I was initially uncertain about whether the ESCs I'm using can be programmed, as there's limited information online, so I'm still exploring that aspect.

For calibration, I set up the ESCs' throttle ranges by following their "beep" patterns. I replaced the old motors with high-quality **EMAX 900 KV** motors and installed **Skywalker 30A-rated ESCs** to improve performance. To enhance stability and prevent drifting during flight, I integrated a **Pixhawk 4 GPS module** with the flight controller. I set up the GPS module as the primary source and recalibrated all the sensors to ensure the flight controller and GPS module readings were aligned. After rebooting the system, I fine-tuned the parameters to optimize performance.

One crucial adjustment I made was setting a minimum satellite requirement (8 satellites) for the GPS module to operate. This ensures that the system only operates when the GPS has enough signal, enhancing flight safety. I also configured the system so that, without a GPS signal, the quadcopter cannot take off, ensuring a safe flight. Additionally, increasing the baud rate of the GPS module boosted performance. I ensured that the **UBX communication protocol** was enabled for proper communication between the flight controller and the GPS module.

After all the setup, I tested the quadcopter in **Position Slow mode**, which I recommend for beginner pilots. This mode helps stabilize the flight by controlling altitude and damping roll, pitch, and yaw, making it easier to handle during flight.

Looking ahead, we plan to acquire a telemetry module to enable **GCS (Ground Control Station)** communication with the flight controller during flight. This would allow us to assign missions and control the drone directly from the GCS without needing a transmitter. The telemetry module will also enable longer-range operations and provide many advanced features that will enhance the capabilities of the drone.



Quadcopter operating at high altitude (50 M from sea level)

Controlled throttle down to ensure safe landing for beginners

Errors encountered and how I tackled them

I learned a valuable lesson about the importance of thoroughly reviewing documentation and specifications. The receiver I'm currently using, the RadioLink R9D, is a 9-channel receiver that supports PWM and SBUS protocols. After some online research, I discovered that the mode of the receiver can be switched between these protocols. However, I mistakenly assumed that the R9D could also support PPM, as I had seen the RadioLink R9DS receiver offering this capability.

As a result, I spent several days troubleshooting why I couldn't proceed with the transmitter calibration setup. I attempted various methods, including enabling multiple RC input options via the parameter settings and verifying MAVLINK communication through the console. Despite my efforts, the issue persisted, and I eventually turned to online resources for guidance.

Then, I discovered the root cause: the R9D receiver does not output PPM signals through channel 1. After switching to a different receiver that supports PPM, I was finally able to resolve the issue and successfully identify the transmitter inputs.

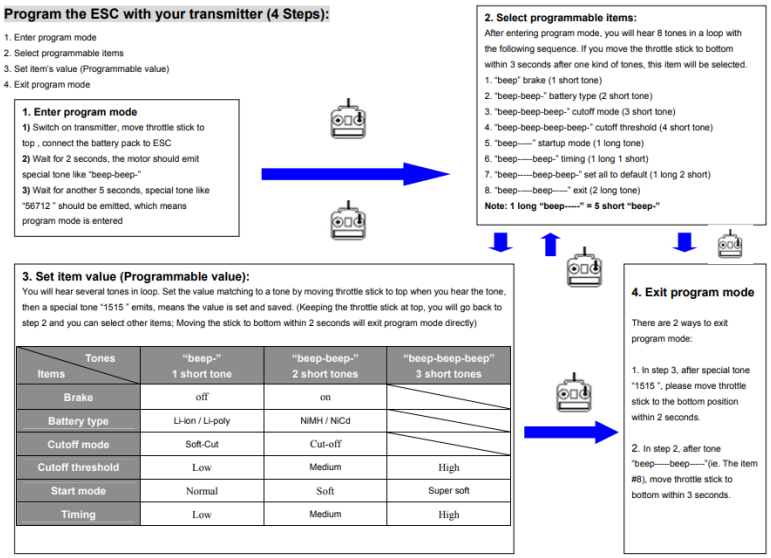
I plan to connect the SBUS port from the flight controller to the receiver to ensure robust and precise communication. SBUS is well-regarded for its clean, noise-immune digital transmission, making it an ideal choice for reliable operation.

With this setup complete, I'm ready to test the actuator configuration and conduct a test flight on a quadcopter prototype to ensure everything functions as expected.

The ESC calibration was another issue. The calibration command for ESCS under the power section did not work properly, so I had to calibrate them manually through a transmitter, and they finally got working. I will have to look into this in more detail later on. Next, the motors spin at a lower rpm. I referred to some advice. I have some chances to improve the rpm via ESC calibration settings, PID tunings, adjustments in the throttle range, Actuator setup, and modifications. I can also recalibrate the ESCS via the calibrate button under the power section and some parameter changes. I will get the quadcopter suitable for flying within no time! Afterwards I will add further improvements to set up the babyshark VTOL that I'm developing under the supervision of the ACCIMT.

Well, everything went really well except for one particular issue…. As usual, the odds are against me, and this time, one of the 4 ESCs is not working properly. Hence, I had to replace it with another working 30 A esc. However, the calibration for that specific ESC differs from the rest, and it's not easy to calibrate the ESC with many beeps. Is it possible to use software to calibrate the ESCs through that to make it easy to work with? I will compare the new ESC I put with the other ESC I use there. I don't have a driver connector to connect the ESC to my PC for communication, but im planning on using a UNO R3 board for this purpose. Most importantly, I believe the best thing I should do now is to understand how ESCS works to tackle this issue properly.

It turns out that the BLHeli software is specifically designed for BLHeli ESCs. After realizing this, I dug deeper into the details and recalibrated the ESCs by carefully following their unique "beep" patterns. Below, I've outlined the detailed calibration process for the ESCs.



Program the ESC with your transmitter (4 Steps):

1. Enter program mode
2. Select programmable items
3. Set item's value (Programmable value)
4. Exit program mode

1. Enter program mode

1) Switch on transmitter, move throttle stick to top , connect the battery pack to ESC
2) Wait for 2 seconds, the motor should emit special tone like "beep-beep-"
3) Wait for another 5 seconds, special tone like "56712 " should be emitted, which means program mode is entered

2. Select programmable items:

After entering program mode, you will hear 8 tones in a loop with the following sequence. If you move the throttle stick to bottom within 3 seconds after one kind of tones, this item will be selected.

1. "beep" brake (1 short tone)
2. "beep-beep-" battery type (2 short tone)
3. "beep-beep-beep-" cutoff mode (3 short tone)
4. "beep-beep-beep-beep-" cutoff threshold (4 short tone)
5. "beep-----" startup mode (1 long tone)
6. "beep-----beep-" timing (1 long 1 short)
7. "beep-----beep-beep-" set all to default (1 long 2 short)
8. "beep-----beep-----" exit (2 long tone)

Note: 1 long "beep-----" = 5 short "beep-"

3. Set item value (Programmable value):

You will hear several tones in loop. Set the value matching to a tone by moving throttle stick to top when you hear the tone, then a special tone "1515 " emits, means the value is set and saved. (Keeping the throttle stick at top, you will go back to step 2 and you can select other items; Moving the stick to bottom within 2 seconds will exit program mode directly)

Tones / Items	"beep-" 1 short tone	"beep-beep-" 2 short tones	"beep-beep-beep" 3 short tones
Brake	off	on	
Battery type	Li-ion / Li-poly	NiMH / NiCd	
Cutoff mode	Soft-Cut	Cut-off	
Cutoff threshold	Low	Medium	High
Start mode	Normal	Soft	Super soft
Timing	Low	Medium	High

4. Exit program mode

There are 2 ways to exit program mode:

1. In step 3, after special tone "1515 ", please move throttle stick to the bottom position within 2 seconds.

2. In step 2, after tone "beep-----beep-----"(ie. The item #8), move throttle stick to bottom within 3 seconds.

One of the unusual issues I encountered was that the ESCs had varying throttle ranges. After identifying this, I recalibrated each ESC individually to ensure they shared the same throttle range. However, I soon faced additional hardware challenges with the motors. Many motors had been used in previous projects and were often damaged. To address this, I adjusted the throttle range of each AUX output in an attempt to achieve stable lift-off. While this approach helped to some extent, I wasn't confident that I was unlocking the full potential of the quadcopter. As a result, I replaced all four motors with 900 KV EMAX motors and paired them with four Skywalker 30A ESCs. The battery remained unchanged, and the results were significantly more promising this time. Finding the ideal motors was challenging, but it was a step forward in solving key hardware issues.

Just as I thought I had overcome the major obstacles, another problem arose. The power module that supplied power to the flight controller via the PDB and battery connector stopped functioning. Upon inspection, I discovered that the step-down buck converter was missing. I attempted to repair it by soldering the components back in place, but the SMD (surface-mounted) parts were too small to handle effectively. After multiple failed attempts, I abandoned the damaged power module and explored alternatives.

One idea was to power the flight controller through the USB input, which initially worked. I then drew power from the main battery, passing it through an L7812CV 12V voltage regulator. While the output measured close to 5V, which matched the flight controller's specifications, the current was insufficient, and the output was unstable, preventing the flight controller from powering up. After discussing the issue with my team, I acquired a proper step-down buck converter. By tuning the output to 5V, I successfully powered the flight controller, setting the stage for my first manual flight.

Flying a drone manually proved to be quite challenging. I switched to stabilized mode, which improved control, but the drone still exhibited mid-flight drifting. Upon further research, I discovered this was a common issue in drones. I implemented a GPS solution to achieve a fully functional, easy-to-control system. By integrating a Pixhawk 4 GPS module with the flight controller and enabling GPS input, I unlocked advanced flight modes such as Position Hold, Altitude Hold, and Follow Me. These modes rely on GPS coordinates, with accuracy improving as the module detects more satellites.

I chose three primary flight modes for my setup: Position Hold, Stabilized, and Position Slow. Position Slow is particularly beginner-friendly, as it manages altitude and smoothens roll, pitch, and yaw controls, making handling much safer and easier.

With the drone now stable, I was ready for my first fully successful flight. However, my mother was concerned about flying near high-voltage lines in our garden. So, I moved to a more spacious area and achieved my first stable, controlled flight—a significant milestone in the project!

(For more details, refer to the Procedure section.)

TO BE CONTINUED